

ML Specification Report — tabular-playground-series-aug-2022

Product Failure Prediction • our from-scratch agent (regularized-LR pool + tree-search v3 + honest LOFO gate)

Figures grounded in the competition's *STATUS.md*, *config.yaml*, *dossier.json*, *experiments.json*, *experiments_tree_v3.json*, and *rules_verdict.json*, with leaderboard and cross-agent figures from the study's frozen three-way score table. Version discipline: this competition ran 2026-07-28 under the pre-v5 (v3-frozen) pipeline, the same configuration as the rest of the frozen 18-competition baseline — Stage 0.5 did not yet exist. The cited *dossier.json* was produced by the retrospective problem-identification sweep of 2026-07-29/30 and was NOT consumed by this run; it is cited only as a descriptive record of the task.

Contents

Product Failure Prediction • our from-scratch agent (regularized-LR pool + tree-search v3 + honest LOFO gate)	1
Overview	1
Data	2
Models & Architecture	3
Training procedures	3
Inference procedures	4
Evaluation & Benchmarking	5
Final-Architecture Re-Run (v5 pipeline, Stage 0.5 in-lane) — 2026-08	5

Overview

The task is to predict product failure (binary, 21.3% positive) from anonymized loading and measurement columns (metric: ROC-AUC, maximize). The structural fact that dominates everything: **train and test contain disjoint product codes** (train A–E, test F–I, zero overlap), so the evaluated quantity is generalization to unseen groups, and validation must be leave-one-group-out. We solve it with a from-scratch pipeline — per-code preprocessing, an LR-vs-GBDT race won decisively by heavily regularized logistic regression, a 60-node *harness_v3* tree search, and an **honest leave-fold-out audit that rejected every fitted-weight blend**. Our champion is an unfitted equal-weight average of two LR models with per-group rank normalization, scoring **OOF AUC 0.591645** (honest LOFO-selection view 0.591204). On the real leaderboard we score **Public 0.58559 / Private 0.59055** (percentile 66.0, rank 644/1889) — second of the three benchmark agents on the private board: AIDE wins at 0.59106, NVIDIA (the local-CV winner at 0.5927) finishes last at 0.58730.

Why it matters. Predicting which manufactured units will fail supports quality control and warranty triage before shipment; as an ML problem it is a clean stress test of whether a pipeline can generalize to product lines never seen in training rather than memorize the ones it has.

Data

Purpose of Data. Predict a binary `failure` label per product unit — a tabular binary-classification Playground episode under disjoint-group covariate shift. **Data Format** is clean **tabular CSV** with 24 predictive columns once `id` and the target are removed: `product_code`, `loading` (continuous), `attribute_0` – `attribute_3`, integer `measurement_0` – `measurement_2` (29/30/25 distinct values), and continuous `measurement_3` – `measurement_17`. **Data Volume** is **26,570 training rows / 20,775 test rows**; positive rate 0.212608 (5,648 failures).

Data Quality has two defining wrinkles. First, the **group structure**: `product_code` partitions the data completely — train holds A–E (5100/5250/5765/5112/5343 rows), test holds F–I (5422/5107/5018/5228) with zero overlap, and `attribute_0` – `attribute_3` are constant within each code, i.e. aliases of the group key with unseen test levels. Second, **monotone missingness**: null rates climb along the measurement index (`measurement_3` 1.43% → `measurement_17` 8.60% in train; 1.58% → 8.38% in test), plus `loading` ~1%. Null rates match closely across splits, so missingness is shift-stable even though the groups are not. The strongest EDA signals:

- **loading is the dominant predictor** — the single named non-measurement continuous column, right-skewed (mean 127.826, median 122.39, max 385.86, std 39.030).
- **Missingness is itself signal**: failure rate is 0.160 when `measurement_3` is missing and 0.254 when `measurement_5` is missing, versus the 0.213 base rate — hence NA indicator flags.
- **Per-group failure rates are tight** (0.2004–0.2273 across A–E), so folds balance naturally without stratification.
- Weak overall signal + heavy shift ⇒ a low AUC ceiling where fold and seed noise rival real gains.

Annotation Guidelines. The label is `failure` $\in \{0, 1\}$; submissions are **continuous scores** ranked by pooled ROC-AUC over all four unseen test codes at once — a rank metric, so calibration and thresholds are irrelevant, but cross-group score-distribution shift directly hurts the pooled score.

Feature Set. All preprocessing is fit **per product code** (codes are disjoint, no target used — no leakage):

Group	Features
Engineered pool (<code>scripts/prep.py</code>)	per-code z-scaled <code>loading</code> + <code>measurement_0..17</code> ; <code>measurement_17</code> imputed per code by HuberRegressor on its top-4 correlated measurements; other NaNs per-code median; per-column NA flags
Root LR set (5)	<code>loading</code> , <code>measurement_17</code> , <code>measurement_2</code> , <code>measurement_3_na</code> , <code>measurement_5_na</code>

Group	Features
Node #17 LR set (7)	the root five + <code>measurement_4</code> , <code>measurement_9</code>
Dropped by design	<code>product_code</code> , <code>attribute_0..3</code> — group-key aliases whose test levels are unseen; target encoding of them forbidden

Splitting strategy. Leave-one-group-out GroupKFold on `product_code` (5 folds, seed 42) — each fold trains on 4 codes and scores on the held-out one, exactly mirroring the train→test regime; shuffled or stratified KFold is forbidden as it would measure within-group ranking, which the leaderboard does not evaluate. The decision metric includes the post-processing: **pooled OOF AUC after per-group rank normalization** (prior-run validated +0.0007; direction confirmed across all configs this run), because the pooled test AUC punishes score-scale shift between unseen groups. The run was **OOF-only with no LB anchor at selection time**; leaderboard figures below come from the benchmark study's later frozen submission. External data is rules-permitted (`rules_verdict.json` , § 7.C) but no whitelisted source is joinable — the schema has no country/date/entity key — so none was used.

Models & Architecture

Purpose of Architecture. A failure-risk ranker that maximizes pooled AUC on unseen product codes. **Architecture Type** is, unusually for this series, **linear-dominant**: under disjoint-group shift with a weak-signal rank metric, a small heavily regularized **logistic regression beats GBDT** (LR 0.591248 vs shallow LGBM 0.58174, confirmed twice), and a rank-space LR+LGB weight search degenerates to $w_{lr} = 1.0$. The champion is an **unfitted equal-weight average of two LR members** — the root ($C = 0.01$, 5 features) and tree-search node #17 ($C = 0.045$, 7 features) — with per-group rank normalization on top. Each member takes the same **Input Format** — a per-code z-scaled numeric matrix with NA flags — of **Input Dimension 5 or 7 features** per row (a 1-D vector; no spatial/sequence structure). **Architecture Description.** Each member is a plain sklearn `LogisticRegression` (`max_iter` = 2000, `random_state` = 42) whose strong L2 penalty ($C = 0.01 / 0.045$) is the capacity control that survives group shift; the ensemble layer is deliberately weightless — a 2-member equal average in per-group rank space, chosen precisely because fitted weights failed the honest audit. **Model Complexity** is tiny: two linear models over ≤ 7 features each — the pipeline's finding is that on this problem, less capacity is more.

Training procedures

Training proceeds as a linear stage, a tree-search layer, and a decisive honest audit, all on the shared LOGO 5-fold split and scored as pooled OOF AUC after per-group rank normalization:

Stage	Configuration	OOF AUC
linear #1	LR $C=0.01$ on prior-run 5-feature set (raw OOF 0.59044)	0.591248
linear #2	shallow LGBM, 35 features, leaves 7 / depth 3	0.58174

Stage	Configuration	OOF AUC
linear #3	LR+LGB rank-space weight search (degenerate $w_{lr} = 1.0$)	0.591248
tree v3, best solo	node #17: LR C=0.045, root five + m4 + m9	0.591447
tree v3, pre-burst blend	node #23 (6 members, Dirichlet weights)	0.591648
tree v3, explore burst	node #52 mega-blend (38 members) — in-sample	0.591955
honest audit → FINAL	unfitted equal-weight pair [root, #17]	0.591645

The tree search (`harness_v3` , 60 nodes evaluated, wall ~7 min) ran lineages LRC / LRFEAT / LGBREG / SEEDBAG / BLEND plus the mandatory explore burst; the root was digit-verified against the linear stage (0.591248), dedup rejections were 0, the cost guard never fired, and the burst sanity gate correctly killed the deep-LGB long shot (#53, 0.551). The **honest audit is the decisive step**: refitting the mega-blend's weights leave-fold-out scored **0.590944 < root 0.591248** — its apparent +0.0007 was pure weight-fitting optimism (+0.001011 measured) — and a loose 22-member equal-weight family scored 0.590510; both rejected. LOFO-selection over 5 candidates picked root+#17 in 4/5 folds (honest score 0.591204 $\approx$ root), so the final choice is the unfitted pair: no fitted weights, downside bounded at noise, variance-reduction rationale.

The **Loss Function** is L2-regularized log-loss (sklearn `LogisticRegression`), with model selection on pooled AUC. The **Optimization Algorithm** is sklearn's convex LR solver (`max_iter` = 2000) — **not** SGD/ADAM — and blend weights, where searched, used Dirichlet sampling ($k = 800$) + coordinate ascent before being rejected in favor of equal weights. There is **no Learning Rate** (N/A — convex solver; the tuned knob is the regularization strength C), **no Learning Rate Scheduler** (N/A), and **no Batch Size** (N/A — full-batch convex optimization). **Training Duration**: the tree search took ~7 min of wall-clock over 60 nodes (an LR solo evaluates in ~0.1 s); total run wall-clock beyond that is not recorded. **Training Memory** was not explicitly capped — $26,570 \times \leq 35$ features is trivial; peak not recorded. **Transfer Learning** is N/A (no pretrained weights); its analogue is **cross-run experience injection** — the 5-feature set and the per-group rank-normalization step were validated in a prior run and re-verified here. **Data Augmentation** is N/A (tabular); the analogues are the NA-flag features and per-code preprocessing.

Reproducibility Standards: fixed seeds throughout (CV seed 42, LR `random_state` 42), the tree-search root retrained and digit-verified against the linear-stage score, the cached solo pool reseeded, and the backtrack log persisted in `experiments_tree_v3.json` . Two findings were written back to the experience library: the first direct AUC quantification of fitted-blend-weight optimism (+0.001 — enough to invert a leaderboard), and LOFO-selection as a zero-cost honest gate.

Inference procedures

Decision Threshold is N/A — ROC-AUC is a rank metric, so we submit continuous scores and never threshold (a prior confirmed twice that CV-tuned thresholds do not transfer). The one post-processing step is first-class here: **per-group rank normalization** applied per test product code, so each unseen group's scores occupy a comparable scale in the pooled AUC. The submission (`submissions/`

`final_root_plus_node17_equalweight.csv` , 20,775 rows) averages the two LR members in rank space. **Inference Duration** was not separately profiled — two linear models over 20,775 rows is sub-second — and **Inference Memory** is negligible.

Evaluation & Benchmarking

Task for Performance Evaluation. Rank the 20,775 test units by failure risk, pooled over four unseen product codes. **Performance Metrics.** ROC-AUC (maximize) is the sole metric; our champion reaches **OOF AUC 0.591645** under LOGO with per-group rank-pp (honest LOFO-selection view 0.591204; root solo 0.591248). The trajectory 0.591248 → 0.591447 → 0.591648 → 0.591955 (rejected) → 0.591645 is the story of a search whose in-sample gains were audited back down: fold AUCs span 0.586–0.600, so every candidate sits inside a ± 0.0005 noise band and the honest ceiling with these features is ~0.591–0.592.

Performance Benchmarking. Per the set's standing convention, **between-agent claims use the leaderboard numbers**; each agent's local CV is its own pipeline's internal view (own scheme, own folds) and is not comparable across agents. From the study's frozen three-way score table:

Agent	Approach	Local CV	Public LB	Private LB	Percentile
my-agent	from-scratch LR pool + tree-search v3 + honest gate	0.591645	0.58559	0.59055	66.0
NVIDIA	reproduce-agent (copies strongest public kernel)	0.5927	0.57920	0.58730	43.0
AIDE	open-source tree-search agent	0.58981	0.58479	0.59106	60.5

AIDE wins the leaderboard (Private 0.59106 vs our 0.59055); we place second (rank 644/1889, percentile 66.0) and lead all three agents on the public split (0.58559). The instructive result is NVIDIA: the **local winner** (0.5927) finishes **last** on both leaderboard splits (0.57920 / 0.58730, percentile 43.0) — a live, between-agent demonstration of exactly the lesson our honest audit encoded within-pipeline: on this weak-signal, group-shifted problem, small local-CV advantages are noise or optimism, and selections made on them do not transfer. Our conservative unfitted pair landed a noise-band-scale gap behind the winning agent (0.59055 vs 0.59106) while refusing every fitted-weight gain the search offered.

Fields marked "not recorded": total run wall-clock beyond the ~7 min tree search; training peak memory; inference duration and memory (not separately profiled). The run itself was OOF-only (no LB feedback at selection time); leaderboard figures come from the benchmark study's frozen score table.

Final-Architecture Re-Run (v5 pipeline, Stage 0.5 in-lane) — 2026-08

All 20 baseline competitions were re-run in an isolated root (`~/benchruns/myagent-rerun`) under one frozen architecture (`docs/rerun_manifest.json`): every lane runs the **full v5 pipeline including the**

Stage 0.5 problem dossier (dossier.json written in-lane before modeling), holds no credentials, and is checked by a per-lane transcript audit. Leaderboard scores for the re-run were produced by the separate credentialed operator scoring step and are recorded in `benchmark_results/rerun_three_way.csv` ; the figures below revise nothing in the sections above.

- **Lane:** ran and was scored.
- re-run local CV (lane `STATUS.md`): **0.591820 pooled ROC-AUC** (per-product rank-normalized, leave-one-product-out over product_code A–E), a 16-member diverse blend; honest nested estimate **0.590949** mean held-out AUC over the 5 unseen-product folds.
- Public / private leaderboard for this re-run (`benchmark_results/rerun_three_way.csv`): **Public 0.58534 / Private 0.58930**; `winner_priv` = **aide** (AIDE 0.59106, NVIDIA 0.58730).